

What Belongs in an Agent’s Context Window

What Import Structure and Issue Text Actually Contribute

NEIL LUNAVAT, Pimpri Chinchwad College of Engineering, India

An agent asked to change one file in an unfamiliar repository cannot read the repository: the median repository here is 286,385 tokens and the files carrying the answer are 2.98% of it. Two signals can choose what it reads: the import structure that graph-based context tools order by, and the words of the issue report. We compare both and their combination on 2,412 change tasks from 1,162 pull requests in 112 Python repositories, scoring each ordering by how much of an answer key it covers per line read, against two keys we never merge: files *co-edited* in the same pull request, and files *defining symbols the change uses*.

In 40.2% of tasks the issue names a key file as *code*, not merely sharing a word with it, and we separate what that is worth from what the rest of the issue is worth. Deleting only those names from the same issue — same task, same prose, same traceback — costs issue-to-path matching 0.110 AUC and the best fusion 0.061, while changing nothing at all on the tasks where there was nothing to delete. Replacing the whole issue with a seed-derived query costs 0.156 on those tasks but only 0.016 on the rest. The two together decompose the apparent value of issue text: filename leakage is causal and large, and issues that name files are also better issues in other ways, each accounting for roughly half. Two further controls each dissolve one half of the key-dependent reversal our uncontrolled tables appear to show: matching the populations the two keys are scored on, and taking the repository rather than the pull request as the unit, which moves the smallest p -value here from 10^{-100} to 10^{-20} with every effect size intact.

What survives is a ranking rather than a mechanism. Fusing a length-aware personalised walk with issue matching is never significantly worse than the best pure ordering under any control, and across 200 random repository splits it is selected on one half in 198 and is best on the other half in 195.

Additional Key Words and Phrases: code search, context retrieval, dependency graphs, software maintenance, empirical software engineering

ACM Reference Format:

Neil Lunavat. 2026. What Belongs in an Agent’s Context Window: What Import Structure and Issue Text Actually Contribute. *ACM Trans. Softw. Eng. Methodol.* 1, 1 (September 2026), 28 pages. <https://doi.org/10.1145/nnnnnnn.nnnnnnn>

1 INTRODUCTION

An agent is handed a bug report and a file to change. Before it can write anything it must decide what else to read, and it cannot simply read everything. In the corpus we study the median repository is 286,385 tokens — counting the Python files in the import graph, excluding tests and non-Python files — and 74.8% of tasks sit in a repository larger than a 128,000-token window, 60.5% in one larger than 200,000. Meanwhile the files that carry the answer are a median 2.98% of the repository. The reduction available is roughly 34×, and every line spent on an irrelevant file is a line not spent on a relevant one. Something has to choose, and the choice is the subject of this paper.

A human faces a softer version of the same problem — attention rather than tokens — and we return to that reading in Section 5. The budget is what makes the question sharp, so we pose it in those terms throughout.

The dominant answer in tool building is the dependency graph. Aider builds a repository map from the call and definition graph; RepoGraph, LocAgent and CoSIL construct code graphs and

Author’s address: Neil Lunavat, Pimpri Chinchwad College of Engineering, India, neillunavat3192@gmail.com.

2026. 1049-331X/2026/9-ART \$15.00
<https://doi.org/10.1145/nnnnnnn.nnnnnnn>

walk them outward from a starting point. The premise is intuitive and rarely stated: what you need to read is near, in the graph, to what you are changing.

There is a second signal, and it is the one a human reaches for first. Asked to fix the login page, a developer thinks of the authentication files — not because `login.py` imports them, which it may well not, but because the words match. That signal sits in the issue report.

We are not the first to notice it. SWE-bench’s own sparse-retrieval baseline is BM25 with the issue as the query [10], Agentless [21] leans on file and directory names, and the agentic systems all read the issue text as part of their loop. What is missing is not the idea that words help. It is a measurement of how far they carry against structure, on a stated answer key, under a stated budget, and with the issue corpus’s habit of naming the answer controlled for. Every system in Table 1 reports end-to-end accuracy against a single key, and none reports that comparison. Nor does any of them report how often its issues name the files it is scored on — a weaker form of the solution leakage that SWE-Bench+ [2] measured, and the form that matters for retrieval, since naming the file is all a retrieval step needs.

This paper compares the two. The comparison sounds simple and is not, because it has no answer until one says what it means for a file to have been “needed”. We use two evidence sources that can be recovered from the repository without human labelling: the files *co-edited* in the same pull request, and the files that *define symbols* the changed code references. Both are defensible. Neither is obviously the right one, and graded as they come they rank the methods differently — which is how we first read our own results.

That reading did not survive. Three controls, each of which we added because the comparison demanded it rather than because a result invited it, between them removed both halves of the key-dependent reversal and most of the measured advantage of issue text. The largest is a leakage control. A benchmark built from real issue reports inherits whatever those reports happen to say, and what they often say is the name of the file to change; when they do, an ordering that matches issue words against file paths is not retrieving the answer so much as reading it. Separating issues that name a key file *as code* from issues that merely share a word with one, and then measuring what the issue adds against an ordering that never sees it, is the experiment this paper is built around.

Research questions

RQ1 How much of a repository is the seed’s two-hop neighbourhood, and how much recall does it buy over a same-sized random sample?

RQ2 Do structural and lexical reading orders rank differently under the two evidence sources, and by how much?

RQ3 How much does conditioning on the seed file add over a query-independent repository prior, and does the answer depend on the evidence source?

RQ4 Do the two signals retrieve the same files, or different ones?

RQ5 Does fusing them produce an ordering that is good under both evidence sources?

RQ6 What fraction of tasks could be solved at all under a given budget — that is, how far is the best ordering we test from one that cannot be improved on?

RQ7 How much of what issue text contributes is the issue naming the answer? When an issue names a key file as code, and when it does not, what does an ordering gain from reading the issue at all?

RQ7 was not in the pre-registration; it grew out of the leakage control, which was. Its answer is the reason the rest of this paper is written the way it is, so we flag its post-hoc status here rather than in a footnote, and Section 3 lists every other analysis added after the freeze.

Contributions

- (1) A head-to-head comparison of import-graph proximity and issue-text retrieval as *reading orders* under a line budget, on 2,412 tasks from 1,162 real pull requests in 112 Python repositories, with both signals scored against the same two evidence sources.
- (2) Evidence that the conclusion of such a comparison is decided as much by the evidence source and by how the corpus is filtered as by the methods. Across twenty contending orderings the two keys induce different rankings, but we show that two of the largest apparent effects are properties of the comparison rather than of the signals: one is filename leakage from the issue text, the other a population mismatch between the two keys. All claims are Holm-corrected with the *repository* as the unit, since pull requests cluster within repositories and treating them as independent inflates significance by many orders of magnitude.
- (3) A quantification of what graph proximity actually contributes: recall over half the repository, at a lift of 1.57 $\times$, with 4.2% precision at two hops.
- (4) A robustness criterion — worst-case rank across evidence sources — under which three of our five fusions beat every pure signal, confirmed on held-out repositories: the fusion selected by this criterion on 56 of our 112 repositories is the best non-oracle ordering on the other 56 under *both* keys. The dividing line between the fusions that succeed and those that fail is not whether path matching is included, since four of the five include it and one of those still fails. It is whether the structural input is *length-aware*. The clean ablation is `rrf_pprpl_issue_path` against `rrf_ppr_issue_path`, which differ only in whether the walk's scores are divided by file length before fusing: worst-case rank 6 against 13.
- (5) A cost accounting in tokens rather than files: a corpus-wide calibration of Python source (8.50 tokens per line in aggregate, median 7.88), and a method-independent ceiling — the share of pull requests whose complete answer fits a given budget at all. An oracle ordering reaches 97.5% of pull requests within 8,000 lines where the best method we test reaches 58.5%. That gap is an upper bound on what better selection could contribute under that budget. It is not an estimate of it: an oracle reads only key files by construction, so fitting a small key into a budget is close to arithmetic.
- (6) A precise statement of what edge direction buys. Reversing a walk is severely harmful — following imports outward beats the same walk run inward by 0.215 AUC on the symbol key over the full corpus ($p_{\text{Holm}} < 10^{-16}$) and by 0.123 on the population matched to carry both keys ($p_{\text{Holm}} < 10^{-14}$), repository as the unit — but *discarding* direction costs nothing, because the undirected walk common in this literature is indistinguishable from the outward one ($p_{\text{Holm}} = 1$ on the full corpus, 0.57 on the matched one). On the co-edited key direction is not merely unhelpful but harmful: all four directed variants rank seventeenth to twentieth, below a same-directory baseline that uses no graph at all, while all four undirected walks rank tenth to fourteenth, above it.
- (7) A decomposition of what issue text contributes, into the part that is the issue naming the answer and the part that is not, by two experiments that bracket the question. Across tasks, each issue-aware fusion is compared against the identical fusion with every issue-derived input replaced by its seed-derived counterpart: the issue is worth +0.156 AUC on the tasks whose issue names a key file as code and +0.016, not significant, on the rest. Within task, the key files' names are deleted from the same issue and nothing else is touched: that alone costs issue-to-path matching 0.110 AUC and the best fusion 0.061, and costs nothing on the tasks where there was nothing to delete. The two are additive. Filename leakage is causal and accounts for 0.061 of the 0.140 gap between the strata; the remaining 0.079 is that

reports which name files are more informative in other ways as well. Neither experiment shows that issue text is useless for code retrieval. Together they show that a benchmark of this kind measures both effects at once, and that separating them costs a regular expression.

- (8) Three controls, two of which each dissolve an effect we had previously reported as a finding, and all of which we recommend as standard practice. A *leakage* control that classifies whether an issue names a key file as code rather than merely sharing a word with it: 40.2% of tasks do, and on the rest issue-to-path matching falls from sixth to thirteenth of the twenty-three orderings on the co-edited key. A *population* control that restricts both keys to the 702 pull requests carrying both, since a co-edited key requires a multi-file pull request while a symbol key does not: on the matched population the best directed walk falls from second to eighth on the symbol key and is no longer distinguishable from path matching. And a *unit* correction: pull requests cluster within repositories, so every test in this paper is run with the repository as the unit, which leaves the effect sizes unchanged and moves the smallest p -value from 10^{-100} to 10^{-20} .
- (9) A replication package with the graphs, tasks, issue-token bags, token counts, orderings, generated tables and analysis scripts.

2 BACKGROUND AND RELATED WORK

Dependency structure as a guide to change. Using program structure to decide what a change touches is old. Impact analysis [4] propagates from a changed element along dependencies; Robillard [19] ranks program elements by the topology of their dependencies; feature-location surveys [7] catalogue structural, textual and dynamic techniques and note that they are usually evaluated in isolation. Ko et al. [11] observed developers navigating largely by identifier names and search rather than by following dependencies.

Change history. Zimmermann et al. [27] and Ying et al. [24] mine co-change from version histories to suggest files that change together. Our co-edited evidence source is the same notion of relatedness, used as ground truth rather than as a method.

Graph-based context for agents. SWE-bench [10] and its successors [3, 25] made repository-scale change tasks a standard benchmark, and a family of systems now assembles context by walking a code graph: Aider’s repository map [1], RepoGraph [15], LocAgent [6] and CoSIL [9]. Agentless [21] is a notable counterexample that leans on file and directory names.

What these systems share is the ground truth they are judged against. Table 1 records it for every system whose evaluation section we read in the published PDF; each entry is quoted or paraphrased from that section, not from an abstract or a secondary description. Where localization accuracy is reported at all, the target is the set of files or functions touched by the gold patch — the SWE-bench convention, and exactly our `co_edited` source. That is the evidence source on which, in our results, structural proximity is the weaker of the two signals on the uncontrolled comparison — an ordering that the leakage control of Section 4.10 then removes.

Leakage from the issue text. That a benchmark’s issue reports can contain their own answers is known. SWE-Bench+ [2] finds that 32.67% of the patches SWE-Agent with GPT-4 gets *passed* had the solution already present in the issue report or its comments, and Agentless [21] reports a manual pass over SWE-bench Lite that turns up problems whose issue description contains the exact ground-truth patch. Both concern the *solution*: the code, or a direct statement of the fix.

Our control concerns something weaker, more common, and sufficient for a different purpose. A retrieval component does not need the fix; it needs only to know which files to open, and an issue that says `core.py` has supplied that without supplying anything else. That is the case for 40.2%

Table 1. What repository-context systems are graded against, read from each paper's own evaluation section (2026-09-23). The upper block is the patch-based convention; the lower block is the concurrent work that departs from it.

System	Venue	Ground truth used for evaluation
<i>Files touched by the gold patch</i>		
LocAgent [6]	ACL 2025	"The affected files or functions ...as identified in the patches, are considered the target locations"
CoSIL [9]	ASE 2025	File- and function-level accuracy on SWE-bench Lite and Verified, i.e. the patched locations
Agentless [21]	FSE 2025	Correct if the edit covers a superset of the ground-truth patch locations, at line, function and file level
LingmaAgent [14]	FSE 2025	Fault locations extracted from the developer patch, at file and function level
SWE-Fixer [22]	ACL 2025	Precision and recall against the defective files of the resolving pull request; BM25 top-30 then a learned reranker
KGCompass [23]	2025	Files and code elements modified by the ground-truth patch, scored by Jaccard overlap
SWE-Adept [8]	2026	Correct only if <i>all</i> locations modified by the ground-truth patch appear in the top- <i>k</i>
SWERank [17]	2025	Every function modified in the pull request is a positive
CodeScout [20]	2026	Reuses LocAgent's patch-processing scripts: modified files, modules and functions
RepoGraph [15]	ICLR 2025	"The percentage of problems where the edit locations match the ground truth patch", at file, function and line level
<i>Something other than the gold patch alone</i>		
GraphLocator [13]	2025	Locations extracted from the fix patch, then dual-annotated and cross-reviewed because patch-derived ground truth "contains missing locations"
SWE-Explore [26]	2026	File and line intervals that successful agents actually <i>read</i> , intersected across several verified trajectories
ContextBench [12]	2026	Human-annotated context traced outward from the patch regions, pruned by a second annotator, validated by an LLM patching from it alone
AgentRetrievalBench [16]	2026	A file is relevant if reading it helps the agent's next step; one of four task types is ripple files beyond an anchor
CodeR [5]	2024	End-to-end on SWE-bench, but its separate localization study uses golden locations marked by the authors

of our tasks, three times the 12.6% whose issue quotes a full path. It is also, as far as we can tell, unreported: none of the systems in Table 1 states the rate for its own evaluation, and the practice we argue for in Section 4.10 is to report it as routinely as the corpus size.

Concurrent work is questioning the key. The patch convention is under active revision, by at least four efforts that appeared while this study was running, and we want to be exact about how close they come. SWE-Explore [26] derives line-level ground truth "from independent agent trajectories that successfully solved the same issue, distilling the specific code regions their solution paths actually consulted", and scores a ranked list "under a fixed line budget" — both the key and the cost model are close relatives of ours. ContextBench [12] annotates gold context by hand, 522,115 lines over 1,136 tasks, and measures recall, precision and efficiency rather than end-to-end resolution. AgentRetrievalBench [16] defines a file as relevant "when reading it would materially help the agent take the next correct step", evaluated under a top-*k* or token budget; one of its four task types, ripple files beyond an anchored edit, is our co-edited source by another name. And GraphLocator [13] keeps the patch key but dual-annotates it, on the stated grounds that patch-derived ground truth misses locations.

We take this as corroboration of the premise rather than as a threat to it: several groups independently concluded in the same year that the patch key is inadequate. What none of them reports is the consequence we measure. Each proposes a better key and evaluates methods against it; we hold two keys already recoverable from existing benchmark data, and show that the choice between them changes the ranking of the methods themselves. The contribution is not the observation that keys differ, and it is not the reversal either: Sections 4.3 and 4.10 show that both halves of that reversal are produced by confounds rather than by the signals. The contribution is the controls that establish this, and the measurement in Section 4.11 of how little issue text adds once the tasks whose issues name the answer are set aside.

Retrieval. BM25 [18] remains the reference lexical ranking function; we use it unchanged, varying only the query.

3 STUDY DESIGN

The corpus, the graph builder, the task definition and the coverage–cost metric were fixed and tagged in version control before any method was run; the present study adds method families and analyses on top of that frozen base. Changes made after that tag, and claims we have retracted, are recorded in the replication package rather than silently absorbed.

The freeze covers the corpus, the graphs, the task definition and the metric. It does not cover the method families: the fusion orderings, the directed walks and the lexical methods queried by issue text were all added afterwards, in response to what the first results showed. Fusion is the paper’s main recommendation and it is an *exploratory* result, not a confirmatory one.

We also use “design freeze” rather than “pre-registration” throughout, and mean the weaker thing. The freeze is a signed tag in our own version control, not a record lodged with an external registry, so it establishes the order in which we did things to a reader who trusts the repository and to nobody else. The protocol in Section 6 has the same status. We report it because stating a design before running it is worth something even unregistered, not because it carries the guarantee that an OSF entry would.

3.1 Repositories and tasks

We draw pull requests from SWE-bench, SWE-bench-Live, SWE-Gym, SWE-bench Pro, Loc-Bench and SWE-rebench, keeping Python repositories with a permissive licence. A pull request that edits k existing non-test source files that are nodes of the import graph yields k tasks: each edited file in turn becomes the *seed*, and the task is to order the remaining files of the repository. This gives 3,123 tasks from 1,873 pull requests in 113 repositories. Of these, 2,412 tasks — 1,162 pull requests in 112 repositories — have at least one key file inside the graph once the seed is removed, and they are the ones scored here; the remaining 711 tasks have an empty key under both sources and are reported but not scored. Those 711 tasks come from 711 distinct pull requests, and 706 of them change a single file: with no second edited file there is no co-edited key, and the symbol key is empty whenever the changed code resolves to nothing outside itself — standard library, third-party packages, or names `jedi` could not resolve. That is 61.2% of all single-file tasks, which is a real limitation of the symbol key and not a filtering choice of ours. Within the scored set, 1,958 tasks (708 pull requests) have a co-edited key and 2,341 (1,156 pull requests) a symbol key.

The median repository contributes 131 graph nodes and the median key is 3 files totalling 1,515 lines, so the task is to find a handful of files among roughly a hundred.

3.2 Import graphs

Graphs are built per base commit with a custom ast walker over the repository at that commit. Nodes are source files; an edge $a \rightarrow b$ means a imports a module resolving to b . Package `__init__` files are not inserted as intermediaries, from `X import Y` resolves to the module, edges are un-weighted, imports inside `TYPE_CHECKING` blocks and inside function bodies are kept, and test files are excluded. Per-file line counts come from the same pass and are the cost unit throughout.

3.3 Two evidence sources

co_edited The other non-test source files changed by the same pull request. This is what a developer in fact touched, and it is independent of the import graph.

symbol Files that define names the changed code references, resolved with a static analyser. This is what the change depends on.

Symbol resolution uses `jedi`’s `goto` with `follow_imports=True`, which reaches a name’s defining file by traversing import statements. This is worth stating plainly because it bounds what the symbol key can show: the key is constructed by walking the import graph, so a method that walks the import graph is being scored against a target built from the same substrate. We report it because it is the natural static notion of “what this code depends on”, not because it is a neutral arbiter between the two method families.

The seed file is removed from both. We deliberately do not merge them: the whole question is whether the choice matters. It does, and the union is not a neutral compromise — the symbol key contributes 7,675 file references against 4,458 for co-editing, so a union largely inherits the structural source’s verdict.

A third source — files actually executed by the tests that the fix repairs — is defined in our protocol and was collected for 113 pull requests before we stopped. We exclude it rather than report it: a source covering under a tenth of the corpus cannot be compared against two that cover all of it, and reporting it alongside them would invite exactly the conflation this paper argues against. The partial collection is in the replication package so that it is visible rather than quietly dropped, and completing it is the obvious extension.

3.4 Reading orders

Every method receives the graph at the base commit, the seed file, per-file identifier bags and the issue text, and returns an ordering of all other nodes. Table 2 lists them by family. Identifier bags are built by splitting every identifier in the file on underscores and camel-case boundaries and lower-casing; the issue text is tokenised identically, so text is matched against code over one vocabulary. BM25 is used with the standard $k_1 = 1.5$, $b = 0.75$ and differs between methods only in the query. Fusion is reciprocal-rank fusion with $k = 60$.

Two points of fairness are worth stating. The structural methods are *given* a correct answer for free — the seed is one of the files the pull request changed. The issue-text methods are given no seed at all. And the metric charges by lines read, so we include length-aware variants (suffix `_p1`, score divided by file length) to separate “ranks the right files highly” from “happens to prefer short files”.

3.5 Metric

An ordering is read from the top until a line budget is exhausted; coverage is the fraction of the key that has been read. We evaluate at budgets $\{250, 500, 1000, 2000, 4000, 8000, 16000, 32000\}$ and report the AUC, the mean coverage over those eight doublings, so that a method must be good

Table 2. Reading orders, by family. Structural methods use the seed and the graph; global methods use neither the seed nor the issue; lexical methods use no graph.

Family	Method	Ordering
structure	bfs_und	breadth-first from the seed, undirected
	hops_lines	by (hop distance from seed, file length)
	ppr_und	personalised PageRank from the seed, undirected
	ppr_und_pl	the same, divided by file length
	ppr_out	the same walk along import edges: what the seed depends on
	ppr_out_pl	the same, divided by file length
	ppr_in	the same walk against import edges: what depends on the seed
global	ppr_in_pl	the same, divided by file length
	pagerank	PageRank of the whole graph; ignores seed and issue
lexical	indegree	number of files importing this one
	bm25_seed	BM25, query = identifiers of the seed file
	path_seed	BM25 over paths, query = the seed's path tokens
	bm25_issue	BM25, query = identifiers of the issue text
	bm25_issue_pl	the same, divided by file length
fusion	path_issue	BM25 over file paths, query = issue tokens
	rrf_hops_path	RRF of hops_lines and path_issue
	rrf_hops_issue_path	RRF of hops_lines, bm25_issue, path_issue
	rrf_pprpl_issue_path	RRF of ppr_und_pl, bm25_issue, path_issue
	rrf_ppr_issue	RRF of ppr_und and bm25_issue
reference	rrf_ppr_issue_path	RRF of ppr_und, bm25_issue, path_issue
	same_dir	files in the seed's directory first
	random	uniformly shuffled: the floor
	oracle	key files only, cheapest first: the ceiling

at both tight and generous budgets. We also report the more directly interpretable quantity: the number of lines that must be read before half the key has been covered.

Orderings with ties are re-drawn three times per task with distinct seeds and the coverage curves averaged.

The budget is denominated in lines because the pre-registered design fixed it that way at freeze-1, and we do not change a frozen metric after seeing results. Lines are nevertheless the wrong unit for a context window, so we measure the conversion rather than assume it: across the 121,075 distinct file versions in the corpus, tokenised with the c1100k_base byte-pair vocabulary, Python source runs to 8.50 tokens per line in aggregate and a median of 7.88 per file, with a 5th–95th percentile range of 5.70–10.90. Absolute counts shift by a small constant factor between tokenizer families; the ratios we report do not depend on the choice. A budget of 8,000 lines is therefore roughly 68,000 tokens, and the eight budgets span roughly 2,100 to 272,000 tokens — from a fraction of a small context window to more than a large one.

3.6 Statistics

Task AUCs are averaged within a pull request first, so a change touching many files does not count many times. Paired comparisons use the Wilcoxon signed-rank test with the Vargha–Delaney A_{12}

Table 3. Where the key files sit relative to the seed, and how big the two-hop ball is. Hop shares are pooled proportions over all key-file references; the ball columns are medians over 2,412 tasks.

Key files at hop distance	1	2	3+	unreachable
co_edited	39.0%	49.1%	9.1%	2.8%
symbol	45.3%	41.4%	9.1%	4.2%
<i>The two-hop ball: 68 files, 21,646 lines, 56% of the repository</i>				
recall 87.5%, precision 4.2%, recall lift over a same-sized random sample 1.57×				
<i>The key: 3 files, 1,515 lines</i>				

effect size, Holm-corrected within each family of comparisons. Confidence intervals are 2,000-resample bootstraps resampling *repositories*, not tasks, so that a few large repositories cannot manufacture significance.

Every rank in this paper is on one scale: position among the twenty-three orderings we score, the oracle included, so the best attainable non-oracle rank is 2. Where we say “the twenty contenders” we mean the subset entered into the paired tests — the twenty-three less the oracle, indegree and bfs_und, which are reported for reference but not tested. Two further orderings, the issue-free fusion twins of Section 4.11, are scored but never ranked: they are controls rather than candidate reading orders, and ranking them would shift every other rank in every table.

The paired tests are reported with the **repository** as the unit: the pull-request scores are averaged within a repository and the test runs over the 112 repositories. Pull requests are not independent draws — they cluster in repositories that differ in size, layout convention and issue-writing culture — and a test that ignores the clustering counts the same evidence many times. The difference is not cosmetic. The same comparison that yields $p < 10^{-100}$ over 1,156 pull requests yields $p < 10^{-20}$ over 112 repositories. The effect sizes are essentially unchanged, which is the point: the conclusions survive, and only the arithmetic of the p -values was ever overstated. We report the pull-request-level tests alongside, in the replication package, so the two can be compared.

Because a co-edited key requires a pull request that touches more than one graph file while a symbol key does not, the two keys do not cover the same pull requests. Wherever we compare across keys we therefore report the comparison twice: over each key's full population, and over the 702 pull requests that carry both (Section 4.3).

4 RESULTS

4.1 RQ1: the two-hop neighbourhood is half the repository

“The files you need are within two hops” is true here: 86.7% of symbol-key files and 88.1% of co-edited files lie within two undirected hops of the seed (Table 3). Taken alone this reads as strong evidence for graph retrieval.

The other half of the statistic undoes it. In the median task the two-hop ball holds 68 of the repository's 131 files and 21,646 lines, which is 51.9% of that repository and more than five times a 4,000-line budget; averaged over tasks rather than read off the median task, the ball is 55.6% of the repository. Its precision is 4.2%. Against a random sample of the same size it lifts recall by a factor of 1.57.

The two-hop neighbourhood is therefore not a filter. It is a statement that the files you need are somewhere in about half the repository — which is exactly the regime in which the *ordering* decides everything and the graph has already said all it has to say.

4.2 RQ2: the two keys rank the methods differently

Table 5 is the uncontrolled comparison, and the one whose reading the rest of this section revises. Throughout the paper, the AUC in a table is a mean over pull requests, while a difference quoted in the text is the mean of the paired per-repository differences, because the repository is the unit every test uses (Section 3.6). The two agree to within a few thousandths and can differ in the third decimal. Under the co-edited key the top five orderings are four fusions and one lexical method; the best purely structural ordering is tenth, and the query-independent priors are twenty-first and twenty-second of twenty-three, below `same_dir` and barely above random. Under the symbol key the table inverts: the top four are three structural orderings and one fusion, and the best lexical ordering is fifteenth.

The individual reversals are large and significant after Holm correction with the repository as the unit. Issue-text path matching beats length-aware personalised PageRank by 0.059 AUC on co-edited files ($A_{12} = 0.403$ for the walk, $p_{\text{Holm}} < 10^{-4}$) and loses to it by 0.100 on symbol files ($A_{12} = 0.671$, $p_{\text{Holm}} < 10^{-8}$).

We rest the reversal on those paired tests rather than on a rank correlation. The correlation is low — Spearman $\rho = 0.24$ over the twenty contenders ($p = 0.31$), and $\rho = 0.37$ over all twenty-two non-oracle orderings ($p = 0.09$) — but with twenty points a non-significant ρ is weak evidence of anything, and its value moves with which methods are included. The Holm-corrected pairwise reversals do not have that weakness.

Two things about Table 5 are *not* safe to read off it, and the next two subsections are about them. The two columns are computed over different pull requests, and the left column is computed over a corpus in which many issues name the answer outright. Each of those turns out to account for one of the two halves of the reversal.

Figure 1 shows the same reversal as coverage curves. Table 6 states it in the unit a context budget is actually spent in: to reach half the co-edited files a reader following length-aware personalised PageRank consumes a median of 6,155 lines, against 4,838 following issue-text path matching and 3,342 following the best fusion. On symbol files the same three numbers are 3,483, 6,638 and 3,957 — the walk and the words trade places, and the fusion is close to the better of the two in both.

4.3 The reversal is smaller on a matched population

A co-edited key exists only for a pull request that changes more than one file in the graph. A symbol key does not need that, and 454 of the 1,156 pull requests with a symbol key change a single file. The two columns of Table 5 are therefore scored on different samples, and single-file changes are exactly the case where a seed's import neighbourhood is most of what there is to find. The comparison has to be repeated on a population that carries both keys.

Table 4 does that, on the 702 pull requests that have both. On the co-edited key nothing moves: no method changes rank by more than one place. On the symbol key the structural family loses most of its advantage. Cost-scaled personalised PageRank along import edges falls from second to eighth (0.491 to 0.417), the undirected form from third to fifth, and `hops_lines` from fourth to sixth, while issue-to-path matching rises from fifteenth to ninth. The three top places under both keys are then held by the same three fusions.

The statistical claim moves with the ranks. On the full corpus a walk along import edges beats path matching on the symbol key by 0.101 ($p_{\text{Holm}} < 10^{-8}$, repository as the unit). On the matched population the same comparison gives +0.008 and $p_{\text{Holm}} = 1$; for `hops_lines` against path matching it gives +0.019 and $p_{\text{Holm}} = 1$. We therefore do not claim that structural proximity beats issue-text retrieval on the symbol key. On a like-for-like sample the two are indistinguishable, and what remains of the reversal is the co-edited half — which the next control also qualifies.

Table 4. The two keys re-scored on the 702 pull requests that carry both, top ten under each. Δ is the change in rank against the full population of Table 5; a positive number means the method does worse once the samples are matched.

Method	Fam.	AUC	#	Δ
oracle	ref	.774	1	–
rrf_pprpl_issue_path	fus	.487	2	–
rrf_hops_issue_path	fus	.480	3	–
rrf_hops_path	fus	.457	4	–
rrf_ppr_issue_path	fus	.445	5	–
path_issue	lex	.435	6	–
bm25_issue	lex	.409	7	–
rrf_ppr_issue	fus	.394	8	–1
bm25_seed	lex	.393	9	+1
hops_lines	str	.389	10	–
oracle	ref	.779	1	–
rrf_pprpl_issue_path	fus	.470	2	–4
rrf_hops_path	fus	.456	3	–2
rrf_hops_issue_path	fus	.454	4	–4
ppr_und_pl	str	.436	5	+2
hops_lines	str	.425	6	+2
rrf_ppr_issue_path	fus	.421	7	–4
ppr_out_pl	str	.417	8	+6
path_issue	lex	.404	9	–6
ppr_und	str	.395	10	+1

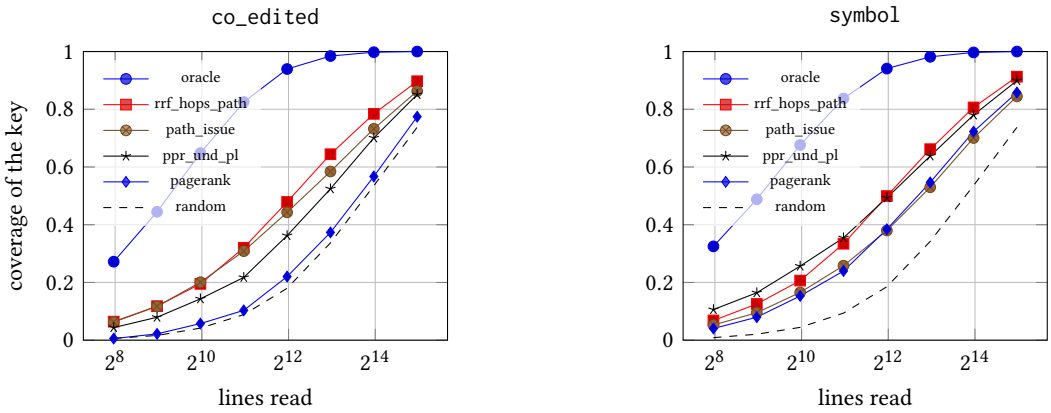


Fig. 1. Mean coverage against lines read, on the full corpus and without either control. The same five orderings change places between the two evidence sources: issue-text path matching tracks the best fusion on co-edited files and falls below the query-independent prior on symbol files, while length-aware personalised PageRank does the reverse. Sections 4.3 and 4.10 show that both halves of this crossing have an explanation that is not about the signals.

Table 5. Mean AUC by evidence source, pull request as the unit, with bootstrap 95% confidence intervals over repositories. The same twenty-three orderings, ranked twice.

co_edited (708 PRs)				symbol (1,156 PRs)			
#	Method	Fam.	AUC [CI]	#	Method	Fam.	AUC [CI]
1	oracle	ref	0.773 [.750,.796]	1	oracle	ref	0.793 [.772,.813]
2	rrf_pprpl_issue_path	fus	0.486 [.455,.517]	2	ppr_out_pl	str	0.491 [.463,.517]
3	rrf_hops_issue_path	fus	0.480 [.448,.510]	3	ppr_und_pl	str	0.489 [.460,.517]
4	rrf_hops_path	fus	0.457 [.425,.489]	4	hops_lines	str	0.477 [.447,.505]
5	rrf_ppr_issue_path	fus	0.445 [.414,.475]	5	rrf_hops_path	fus	0.476 [.449,.502]
6	path_issue	lex	0.434 [.405,.465]	6	rrf_pprpl_issue_path	fus	0.474 [.445,.501]
7	bm25_issue	lex	0.407 [.379,.435]	7	ppr_out	str	0.458 [.430,.487]
8	bm25_seed	lex	0.393 [.362,.425]	8	rrf_hops_issue_path	fus	0.457 [.429,.483]
9	rrf_ppr_issue	fus	0.393 [.363,.425]	9	ppr_und	str	0.446 [.417,.475]
10	hops_lines	str	0.389 [.354,.423]	10	bfs_und	str	0.420 [.391,.447]
11	ppr_und_pl	str	0.371 [.338,.403]	11	rrf_ppr_issue_path	fus	0.420 [.392,.448]
12	path_seed	lex	0.371 [.338,.404]	12	pagerank	glo	0.410 [.381,.439]
13	bfs_und	str	0.362 [.330,.395]	13	indegree	glo	0.398 [.371,.425]
14	ppr_und	str	0.357 [.325,.391]	14	rrf_ppr_issue	fus	0.390 [.361,.421]
15	bm25_issue_pl	lex	0.354 [.318,.389]	15	path_issue	lex	0.382 [.352,.412]
16	same_dir	ref	0.329 [.298,.360]	16	path_seed	lex	0.365 [.336,.392]
17	ppr_out_pl	str	0.327 [.296,.358]	17	bm25_issue_pl	lex	0.355 [.324,.386]
18	ppr_out	str	0.312 [.282,.344]	18	bm25_issue	lex	0.340 [.311,.368]
19	ppr_in_pl	str	0.310 [.279,.341]	19	bm25_seed	lex	0.336 [.305,.365]
20	ppr_in	str	0.308 [.278,.339]	20	same_dir	ref	0.320 [.290,.349]
21	pagerank	glo	0.271 [.240,.301]	21	ppr_in_pl	str	0.277 [.248,.303]
22	indegree	glo	0.271 [.240,.301]	22	ppr_in	str	0.269 [.242,.295]
23	random	ref	0.248 [.220,.277]	23	random	ref	0.248 [.220,.276]

Table 6. Median lines that must be read before half, then all, of the key is covered. Lower is better.

Method	to half the key		to all of it	
	co_edited	symbol	co_edited	symbol
oracle	524	440	1,150	1,349
rrf_pprpl_issue_path	3,342	3,957	7,213	9,563
rrf_hops_path	3,962	3,715	8,073	9,636
ppr_out_pl	8,655	3,924	15,396	12,386
ppr_und_pl	6,155	3,483	11,644	11,303
hops_lines	5,620	4,080	11,454	11,582
path_issue	4,838	6,638	11,072	14,504
bm25_issue	5,012	7,451	10,926	16,699
random	14,557	14,713	21,026	22,643

4.4 Direction: reversing a walk is costly, discarding it is free

The walks above are undirected, as they are in most of this literature. An import edge has a direction, so we add both orientations of the personalised walk, length-aware and not.

The result is asymmetric, and the asymmetry is the finding. Running the walk *along* import edges — towards what the seed depends on — places ppr_out_pl second of twenty-three on the

symbol key at 0.491. Running it *against* them collapses to ppr_in_pl at 0.277, twenty-first. The like-for-like gap is 0.123 on the matched population and 0.215 on the full one, in both cases the largest separation between two structural orderings anywhere in our results ($p_{\text{Holm}} < 10^{-14}$, repository as the unit).

But the undirected walk loses almost nothing. On the symbol key ppr_und_pl scores 0.489 against ppr_out_pl's 0.491, a difference of 0.002 that is not significant on the full corpus ($p_{\text{Holm}} = 1$) and reverses in sign, slightly, on the matched one. So the practice this literature has settled on — throw the direction away — is not the mistake it might appear to be from the spread of the table. Walking backwards is the mistake.

On the co-edited key direction is worse than useless. All four directed variants rank seventeenth to twentieth, below same_dir, a baseline that orders files by directory and consults no graph at all; all four undirected walks rank tenth to fourteenth, above it. Committing to an orientation costs 0.044 to 0.061 AUC there ($p_{\text{Holm}} < 10^{-3}$).

The explanation is the same one that qualifies the symbol key throughout. That key is built by resolving names with jedi, which follows imports in exactly one direction, so a walk in that direction is measuring something close to the key's own construction. The co-edited key records a human decision about what belonged in one change, and no orientation of the import graph recovers it well.

4.5 RQ3: what the seed is worth depends on the source too

A query-independent PageRank prior — a ranking that never looks at the seed or the issue, and simply says which files matter in this repository — reaches 0.410 on symbol files against 0.491 for the best length-aware personalised walk, which does look at the seed. That is 83% of the personalised score in absolute terms, or 67% of its margin over a random ordering, which is the fairer of the two readings and the one recommendation (4) quotes. It is far above random (0.248; $A_{12} = 0.758$, $p_{\text{Holm}} < 10^{-16}$). On that evidence source, much of what the personalised walk delivers is a repository prior.

Under the co-edited key the same prior scores 0.271 against a random floor of 0.248. The difference is significant but negligible ($A_{12} = 0.552$, $p_{\text{Holm}} = 0.001$), and the personalised walk beats it by 0.101 ($A_{12} = 0.666$). Knowing which file is being changed therefore matters a great deal for predicting what else was changed, and rather little for predicting what the code calls, where knowing the repository's hubs is nearly as good.

This is worth stating plainly, because the opposite claim is easy to make from a single source: whether “the graph knows what you are working on” appears true or false is itself a function of the answer key.

4.6 RQ4: the two signals name different files

The reversal is not two methods disagreeing about the order of one list. At $k = 10$ the top-10 sets of personalised PageRank and issue-text BM25 have a mean Jaccard overlap of 0.187, and of personalised PageRank and issue-text path matching only 0.102; at $k = 50$ these rise to 0.508 and 0.412. For comparison, personalised PageRank and the query-independent prior overlap at 0.364 at $k = 10$: a seed-conditioned walk resembles the repository prior more than it resembles a lexical ranking.

Table 7 follows individual key files. At $k = 20$, 24.4% of co-edited files are found only by the lexical family and 9.6% only by the structural family; on symbol files the asymmetry flips (19.9% structure-only against 16.5% words-only). Under either source about a sixth of key files are found by neither family at $k = 20$, which is where the remaining headroom to the oracle lives.

Table 7. How each key file is reached within the top k of the structural family (ppr_und, hops_lines) and the lexical family (bm25_issue, path_issue).

Source	k	both	structure only	words only	neither
co_edited	10	32.2%	11.4%	30.0%	26.4%
co_edited	20	50.7%	9.6%	24.4%	15.3%
co_edited	50	76.7%	5.9%	13.4%	4.0%
symbol	10	28.5%	23.3%	20.4%	27.8%
symbol	20	47.4%	19.9%	16.5%	16.2%
symbol	50	72.9%	11.5%	9.8%	5.9%

4.7 RQ5: fusion is the only ordering that survives both sources

If the evidence source is not known in advance — and for a tool shipped to users it is not — the quantity to optimise is the worst a method does across sources. We report that two ways. Worst-case *rank* is easy to read but depends on the competitor set: adding our four directed walks reshuffled everything below them, and a criterion that moves when other people’s methods are added is a poor one to recommend. Worst-case *shortfall* — the largest AUC gap to the best non-oracle ordering under either key — says the same thing in units nobody else’s entries can change. Table 9 gives both and sorts by the second. The two agree on the family and disagree on its best member: rrf_pprpl_issue_path has the smaller shortfall (0.017 against 0.029) while rrf_hops_path has the better worst rank (5 against 6). We recommend the former, because the shortfall criterion, the matched population (Section 4.3) and the held-out splits (Section 4.8) all select it and only the rank criterion does not. Three of the five fusions beat every pure signal. The two that do not — rrf_ppr_issue and rrf_ppr_issue_path — are not distinguished by whether they include path matching, since four of the five do and one of those still fails. They are the two built on the unscaled ppr_und. The three that survive all fuse from a *length-aware* structural input: either ppr_und_pl, whose scores are divided by file length, or hops_lines, which breaks ties between equidistant files by preferring the shorter one. The two differ in how they use length and agree in that they use it at all.

One pair isolates that property exactly. rrf_pprpl_issue_path and rrf_ppr_issue_path fuse the same three lists and differ only in whether the walk’s scores are divided by file length first; their worst-case ranks are 6 and 13, and the difference is +0.042 AUC on co-edited files and +0.050 on symbol files on the matched population ($p_{\text{Holm}} < 10^{-11}$, repository as the unit). rrf_ppr_issue differs from its length-aware counterpart in two ways at once, so we rest the claim on the clean pair rather than on the three-of-five tally.

The insurance is close to free. Over the full corpus rrf_hops_path gains 0.067 AUC on hops_lines on co-edited files ($A_{12} = 0.602$, $p_{\text{Holm}} < 10^{-11}$) and 0.003 on symbol files ($p_{\text{Holm}} = 1$); against ppr_und_pl it gains 0.087 ($p_{\text{Holm}} < 10^{-13}$) and loses 0.008 ($p_{\text{Holm}} = 1$). The lexical signal adds a large amount where structure is weak and costs nothing where structure is strong. Section 4.11 asks what, in that lexical signal, is doing the work.

4.8 The fusion holds on repositories it was not chosen on

Both the fusion recipe and the worst-rank criterion that selects it were settled after we had seen the full results. That makes the recommendation exploratory however large its margin, so we split the corpus. A random 56 of the 112 repositories form a selection half; the remaining 56 are set aside. We apply the criterion mechanically to the selection half — among the fusions, take the one whose worst rank across the two keys is best — and then report that ordering, unexamined, on the

Table 8. 200 random 56/56 splits of the repositories. “Selected” counts the splits in which the worst-rank criterion picked this fusion on the selection half; “best” counts those in which it was then the best non-oracle ordering on the held-out half; “gap” is its mean AUC shortfall behind the best held-out ordering.

Method	selected	co_edited		symbol	
		best	gap	best	gap
rrf_pprpl_issue_path	198/200	195/198	.002	198/198	.000
rrf_hops_issue_path	2/200	0/2	.016	0/2	.024

other half. Splitting by repository rather than by pull request keeps every task of a repository on one side, which is what the clustering in Section 3.6 requires.

One split would not settle much: with five candidates and a winner that is already best overall, a single draw is close to a foregone conclusion. So we repeat it 200 times (Table 8). `rrf_pprpl_issue_path` is selected in 198 of the 200 splits, and in 195 of those it is the best non-oracle ordering on the co-edited key of the half that did not select it, 198 of 198 on the symbol key. Its mean shortfall behind the best held-out ordering is 0.002 AUC on co-edited files and 0.000 on symbol files. The two splits that select `rrf_hops_issue_path` instead do not confirm it.

This does not make the fusion family a discovery — it was chosen by us, from five candidates, on these data — but the specific recommendation within that family no longer rests on the repositories that produced it.

`rrf_pprpl_issue_path` is also insensitive to the one free parameter we chose without testing. Reciprocal-rank fusion’s k was fixed at the customary 60; re-running the whole pipeline at $k = 10$ and $k = 200$ moves its AUC by at most 0.007 on the co-edited key and 0.010 on the symbol key, an order of magnitude below the differences this section turns on.

4.9 RQ6: what a budget rules out, and what it does not

A coverage number says how much of the key a method found. It does not say whether finding all of it was possible. If the needed files together exceed the budget, no ordering of them fits and no downstream system can succeed, however capable: a model cannot use a file it was never given. That bound is a property of the task and the budget alone, so it can be measured once and applied to every method at once — and, usefully here, without running a model.

Table 10 reports it. The complete answer fits a 32,000-token budget for 90.3% of pull requests under the co-edited key and 87.3% under the symbol key, and a 64,000-token budget for over 95% of both. The task is not, in general, impossible at realistic budgets.

We are careful about what this licenses. The oracle reads only key files, so that it fits a small key into a small budget is close to arithmetic: the ceiling shows the keys are small, not that the remaining distance is reachable from the signals available. What it does rule out is the competing explanation — that current methods score as they do because the answers do not fit — and that is worth ruling out, because it is the explanation a reader reaches for first. The gap between 58.5% and 97.5% is therefore an upper bound on what better selection could add, not an estimate of it.

Against that ceiling, current orderings fall a long way short. Table 11 counts the pull requests each method takes to *complete* coverage at 8,000 lines, a budget at which an oracle succeeds on 97.5% of them. Two medians in this paper look as though they disagree and do not: the median *task* needs 1,150 lines read before the co-edited key is fully covered (Table 6), while Table 10 says 54.2% of *pull requests* fit inside 8,000 tokens. Averaging a pull request’s tasks first gives a median of 991 lines, and at the median 7.88 tokens per line that is about 7,800 tokens — just inside the budget, which is why just over half of them fit. The best method on the co-edited key,

Table 9. Robustness across the two evidence sources, by worst rank and by worst shortfall. Ranks are those of Table 5, oracle included, so the best attainable worst rank is 2. “Gap” is the larger of the two AUC distances to the best non-oracle ordering under each key, and does not depend on which other methods were scored. Sorted by gap; lower is better on both. The three fusions built on a length-aware structural input beat every pure signal on either measure; the two built on the unscaled walk do not.

Method	Fam.	co_edited	symbol	worst	gap
rrf_pprp1_issue_path	fus	2	6	6	.017
rrf_hops_path	fus	4	5	5	.029
rrf_hops_issue_path	fus	3	8	8	.034
rrf_ppr_issue_path	fus	5	11	11	.071
hops_lines	str	10	4	10	.097
rrf_ppr_issue	fus	9	14	14	.100
path_issue	lex	6	15	15	.108
ppr_und_pl	str	11	3	11	.114
bfs_und	str	13	10	13	.124
path_seed	lex	12	16	16	.126
ppr_und	str	14	9	14	.128
bm25_issue_pl	lex	15	17	17	.135
bm25_issue	lex	7	18	18	.150
bm25_seed	lex	8	19	19	.155
ppr_out_pl	str	17	2	17	.159
same_dir	ref	16	20	20	.171
ppr_out	str	18	7	18	.173
ppr_in_pl	str	19	21	21	.214
pagerank	glo	21	12	21	.215
indegree	glo	22	13	22	.215
ppr_in	str	20	22	22	.222
random	ref	23	23	23	.242

Table 10. Share of pull requests whose *entire* key fits within a token budget. This is a ceiling: no ordering can exceed it, and no agent can act on files outside it.

Token budget	co_edited (%)	symbol (%)
4,000	34.1	31.5
8,000	54.2	50.8
16,000	75.1	72.5
32,000	90.3	87.3
64,000	96.9	95.5
128,000	99.5	99.4
200,000	99.9	99.9
1,000,000	100.0	100.0

rrf_hops_issue_path, reaches 58.5%; the best on the symbol key, hops_lines, reaches 51.0%. The ceiling is partly definitional — an oracle that reads only key files must win, so fitting a small key inside a large budget is close to arithmetic — and we therefore do not read the roughly forty-point gap as headroom that better ordering could be expected to recover. Its value is the converse reading, which the definitional part does not weaken: at 8,000 lines the budget is almost never

Table 11. Share of pull requests taken to *complete* coverage within 8,000 lines. The oracle row is the ceiling of Table 10 expressed in the frozen unit. The reversal of Table 5 reappears under this stricter criterion.

Ordering	co_edited (%)	symbol (%)
oracle	97.5	95.9
rff_hops_issue_path	58.5	49.3
rff_pprpl_issue_path	56.5	49.1
rff_hops_path	54.5	50.3
rff_ppr_issue_path	53.4	45.0
bm25_issue	51.8	36.9
rff_ppr_issue	50.6	44.6
bm25_seed	48.8	36.6
path_issue	48.4	36.9
hops_lines	48.1	51.0
ppr_und_pl	45.1	50.0
ppr_und	45.0	50.1
bm25_issue_pl	39.8	33.9
path_seed	39.6	36.3
bfs_und	36.6	40.1
ppr_out_pl	35.4	50.1
ppr_out	34.5	49.7
ppr_in	34.0	22.8
ppr_in_pl	33.9	22.3
pagerank	31.9	43.4
indegree	28.2	38.9
same_dir	27.8	24.2
random	15.3	14.3

the binding constraint, so a failure to cover the key is a failure of the ordering rather than of the budget.

Two things are worth noting. The reversal survives the stricter criterion: `bm25_issue` loses 14.9 points and `path_issue` 11.5 points moving from the co-edited key to the symbol key, while the outward walks gain the most (`ppr_out` 15.2 points, `ppr_out_pl` 14.7) and `pagerank` gains 11.5. The gain is not a property of the structural family as such: the two inward walks lose 11.2 and 11.7 points, about as much as `path_issue`'s 11.5 and in the same direction. It is a property of walking the direction the key was built along, not of using the graph. And the fusion orderings again sit at or near the top of both columns, which is the same conclusion RQ5 reached by AUC, reached here by a measure that ignores partial credit entirely.

4.10 Do the lexical methods just copy paths out of the issue?

Issue reports often contain stack traces, and a stack trace names files. If the lexical advantage came from reports quoting the paths of the files that had to change, it would be an artefact of the benchmark rather than a property of language.

Largely it does, and the effect is big enough that we report it as a result rather than as a reassurance.

The full path of a key file appears verbatim in the issue text for 12.6% of tasks, and a bare file stem for 48.5%. Path BM25 tokenises a path into lowercased subtokens, so a stem is enough to match and the full-path control is far too weak. But a bare stem is also too strong a criterion on its own:

an issue about “models” and a key file called `models.py` share a word because a project’s prose and its filenames draw on one domain vocabulary, and using that overlap is retrieval, not leakage. We therefore classify every stem occurrence by how it is written. A mention is *explicit* if it appears as code — `core.py`, `simbad/core`, a dotted module path, inside backticks, or inside a fenced or indented block — and *incidental* if the stem occurs only as an ordinary English word. Explicit mentions cover 40.2% of tasks. A further 8.3% have incidental matches only, the two together making up the 48.5% of tasks with any stem at all.

That gives three nested strata: issues quoting no full key path (2,108 tasks), issues naming no key file as code (1,442 tasks), and the strictest set containing no key stem at all (1,243 tasks). We treat the middle one as primary. It removes the leakage channel while leaving the vocabulary signal in place, and it does not select as heavily for vague issues as the strictest does.

Table 12 applies this control *together with* the population control of Section 4.3, recomputing within each stratum the set of pull requests that carry both keys. Applied on its own, the leakage control would leave the symbol columns full of the single-file pull requests that the matching control exists to remove, and a claim that survives each control separately need not survive both. Reading the strata jointly is the only version of this table we regard as evidence.

On co-edited files, path matching falls and structure does not follow it down. On the full corpus `path_issue` is sixth of the twenty-three orderings. Under the weak full-path control it barely moves, falling from 0.434 to 0.408 and holding sixth. Under the primary control it falls to 0.363 and *thirteenth*; under the strictest, to 0.339 and *fourteenth*. `hops_lines` — pure structure, no issue text — rises from tenth to sixth in both. Because the middle and the strictest strata agree, the finding is not an artefact of over-correcting: it does not depend on discarding incidental vocabulary overlap.

The direction of that change is clear; its significance is weaker than the ranks suggest, and we report the tests rather than the gap. `hops_lines` over `path_issue` on co-edited files is -0.007 under the full-path control ($p_{\text{Holm}} = 1$), $+0.042$ under the primary control ($p_{\text{Holm}} = 0.40$) and $+0.056$ under the strictest ($p_{\text{Holm}} = 0.02$). Only in the strictest stratum does pure structure beat path matching outright. The honest statement for the primary stratum is that the two become *indistinguishable*, which is already the interesting claim: the ordering that appeared to be the best single signal on this key stops being distinguishable from one that never reads the issue, once the issues that name the answer are removed. These strata are smaller than the corpus — 531 pull requests across 109 repositories in the primary stratum — so the failure to separate them is in part a matter of power, and we do not read it as evidence of equivalence.

On symbol files, length-aware walks lead in every stratum. `ppr_und_p1` and `hops_lines` lead under both real controls, second and third on the oracle-inclusive scale, at 0.445 and 0.435, with the best fusion fourth at 0.430. The gap is not significant ($+0.017$ for `ppr_und_p1`, $p_{\text{Holm}} = 1$), so this is a place where the fusion stops leading rather than a place where it loses.

The seed is not the explanation. The `path_seed` control separates “issue text” from “path matching” directly: the same path-BM25 ranking, queried by the seed’s own path instead of the issue, scores 0.371 on co-edited files against 0.434 for the issue query ($+0.064$, $p_{\text{Holm}} < 10^{-14}$), and the two are indistinguishable on symbol files ($+0.018$, $p_{\text{Holm}} = 0.30$). The comparable leakage figures for the seed itself are 10.3% full path and 35.1% stem.

4.11 RQ7: what does reading the issue actually add?

The previous subsection removes tasks. That answers whether a ranking survives the removal, but it cannot say what the issue text contributes, for two reasons. Every ordering is affected by which tasks are dropped; and the tasks that are dropped are not a random sample of issues. A report

Table 12. Both controls at once: AUC and rank (in brackets) under the three nested leakage strata, each recomputed on the pull requests within that stratum that carry both keys. Ranks are over all twenty-three orderings with the oracle included, the scale of Table 5.

Method	no full path		no explicit mention		no path or stem	
	co_edited	symbol	co_edited	symbol	co_edited	symbol
rrf_pprpl_issue_path	.468 (2)	.456 (2)	.434 (2)	.430 (4)	.417 (3)	.414 (5)
rrf_hops_path	.441 (4)	.445 (3)	.410 (4)	.421 (6)	.392 (4)	.404 (7)
ppr_out_pl	.325 (17)	.419 (7)	.335 (16)	.427 (5)	.331 (16)	.423 (4)
ppr_und_pl	.374 (11)	.440 (5)	.378 (10)	.445 (2)	.373 (10)	.441 (2)
hops_lines	.390 (9)	.427 (6)	.394 (6)	.435 (3)	.387 (6)	.429 (3)
path_issue	.408 (6)	.383 (11)	.363 (13)	.343 (19)	.339 (14)	.320 (20)
bm25_issue	.397 (7)	.357 (15)	.384 (7)	.346 (16)	.385 (9)	.346 (17)
pagerank	.274 (21)	.356 (16)	.281 (21)	.360 (14)	.285 (21)	.361 (13)
<i>pull requests</i>	655	655	531	531	487	487

containing a stack trace is plausibly longer, more technical and more tractable than a one-line feature request, so a comparison across leakage strata compares kinds of issue as well as degrees of leakage. We answer the question twice, once in each direction, and the two answers together decompose the effect.

Across tasks: an issue-free twin. For each of our two leading fusions we construct a twin in which every issue-derived input is replaced by its seed-derived counterpart. `rrf_pprpl_issue_path`, which fuses a length-aware walk with BM25 over file contents and BM25 over file paths, both queried by the issue, is paired with `rrf_pprpl_seedpath`, which fuses the same walk with the same two BM25 rankings queried by the *seed file's own path and contents*. The twin sees the same graph, blends the same number of lists through the same rank fusion, and runs the same retrieval machinery. It simply never reads the issue. The difference is what the issue is worth, net of the “a fusion just gets more input” explanation the fusion result otherwise invites.

Over the corpus the issue is worth +0.069 AUC on the co-edited key and +0.055 on the symbol key, both strongly significant (Table 13). Split by the leakage classification, that becomes +0.156 and +0.121 on the 40.2% of tasks whose issue names a key file as code, against +0.016 and +0.012 on the rest, where neither survives correction; requiring the stem to be absent as well takes it to +0.003 and −0.002. The progression is monotone across five strata and holds for both pairs and both keys.

Because these are null results we report what they exclude rather than treating them as zeros. The bootstrap interval on the no-mention stratum is [0.000, 0.031] on the co-edited key and [−0.003, 0.027] on the symbol key, so the data are consistent with the issue being worth up to about 0.03 AUC there, and inconsistent with its being worth anything close to the 0.156 it is worth where it names a file. Taken alone this comparison would say that the issue is worth almost exactly what it names — and taken alone it would be overstating, because it is confounded with issue quality.

Within task: redaction. So we did the other experiment. For every task we take the *same* issue and delete only the occurrences of its key files' names that our classifier calls explicit, leaving the prose, the traceback, the surrounding directory names and the length untouched. Scoring the same task on both versions isolates the names themselves: the issue, the writer, the project and the difficulty are all held fixed. Only the stem and any immediately following `.py` are removed, not the whole path, so that deleting `astroquery/simbad/core.py` does not also delete the directory

Table 13. What the issue is worth: each fusion minus its issue-free twin, in AUC, with the repository as the unit and the population matched to carry both keys. Positive means the issue-aware ordering is better. The first row is not a control but the complement of the fourth: it is the tasks whose issue *does* name a key file as code. Starred differences survive Holm correction at 0.05.

Issue stratum	repos	rrf_pprpl_issue_path		rrf_hops_path	
		co_edited	symbol	co_edited	symbol
names a key file as code	105	+0.156*	+0.121*	+0.148*	+0.114*
every task	111	+0.069*	+0.055*	+0.053*	+0.038*
no full path quoted	111	+0.047*	+0.036*	+0.033*	+0.022*
no explicit mention	109	+0.016	+0.012	−0.001	−0.009
no stem at all	107	+0.003	−0.002	−0.008	−0.018

names the report is entitled to mention. The residual directory signal is a weaker channel that survives this control, and we state it as a limitation rather than claim otherwise.

The result is in Table 14. On the 970 tasks whose issues name a key file, removing just the names costs path_issue 0.110 AUC on the co-edited key and 0.092 on the symbol key; it costs the best fusion 0.061 and 0.050. Every one of these survives Holm correction with the repository as the unit, and the bootstrap intervals are narrow: for path_issue on co-edited files it is [0.090, 0.131], and the widest of the eight spans 0.041. On the 1,442 tasks where the redactor found nothing to remove, the difference is at most 0.002 in every one of the eight cells and significant in none, which is the sanity check the design demands — the pipeline does nothing when there is nothing to delete.

The decomposition. Both experiments run on the same matched population with the repository as the unit, so their answers add up, and together they say something neither says alone. On the tasks whose issue names a key file, the issue as a whole is worth +0.156 AUC to the best fusion on the co-edited key and +0.121 on the symbol key. Of that, the names account for +0.061 and +0.050, and the rest of the issue for +0.095 and +0.071, every term significant under Holm correction. On the tasks whose issue names nothing, the issue as a whole is worth +0.016 and +0.012, and neither is significant.

So both effects are real and neither subsumes the other. Filename leakage is causal and large: path matching loses a fifth of its score on the affected tasks when the names go, and the fusion loses 0.061. But issues that name files are *also* better issues in other ways — the non-name part of such an issue is worth +0.095, against +0.016 for an entire issue that names nothing. A reader of the stratified table alone would put the whole +0.140 stratum difference down to leakage. The redaction says +0.061 of it is leakage and the remaining +0.079 is a property of which issues get written that way.

What this licenses. It does not show that issue text is useless for code retrieval; developers plainly use it, and an ordering that exploits a real report naming a real file exploits something that exists. What it shows is that a benchmark assembled from issue reports of this kind measures two things at once, and that the two can be separated for the cost of a regular expression. Any evaluation on SWE-bench-derived data that reports a gain from issue-aware retrieval and does not stratify is reporting their sum. That, rather than the ranking of our own methods, is the finding we would most like the field to take up.

Table 14. Redaction, within task: AUC on the issue as written, and the change when the key files' names are deleted from that same issue. Repository as the unit on the matched population, as elsewhere; Holm-corrected, and a star marks $p < 0.05$. The lower block is the control — where the redactor removed nothing, it must change nothing, and it does not.

Method	co_edited		symbol	
	AUC	redacted	AUC	redacted
<i>names removed (970 tasks)</i>				
path_issue	.550	−0.110*	.498	−0.092*
bm25_issue	.452	−0.009*	.408	−0.006*
rff_hops_path	.529	−0.072*	.515	−0.059*
rff_pprpl_issue_path	.566	−0.061*	.537	−0.050*
<i>nothing removed (1,442 tasks; the difference must be nil)</i>				
path_issue	.363	−0.000	.347	−0.002
bm25_issue	.376	−0.000	.341	−0.000
rff_hops_path	.415	+0.001	.426	−0.001
rff_pprpl_issue_path	.434	+0.002	.433	+0.000

4.12 Sensitivity: does the uncontrolled ranking survive in large repositories?

The corpus skews small — the median repository contributes 131 graph nodes — so the reversal has to be shown not to be an artefact of repositories one can almost read whole. Table 15 repeats the ranking within quartiles of repository size (quartile boundaries 73, 131 and 246 nodes; the largest repository has 2,487). The median of 131 graph nodes and the median of 286,385 tokens per repository are consistent: they imply roughly 2,200 tokens, or some 260 lines, per file, which is an ordinary Python module.

The pattern of Table 5 holds in every quartile, so the uncontrolled ranking is not an artefact of repositories one can almost read whole. This analysis runs on the full corpus without either control, so it says nothing about whether the qualifications of Sections 4.3 and 4.10 are size-dependent; we report it as a robustness check on Table 5 alone. The gap does not widen monotonically with size: the co-edited-minus-symbol rank gap for ppr_und_pl runs 9, 5, 9, 10 across the quartiles and for hops_lines 5, 1, 3, 3, so the right claim is stability, not a trend. In the largest quartile, issue-text path matching is sixth on co-edited files and fifteenth on symbol files, while length-aware personalised PageRank is twelfth and second respectively. The query-independent prior degrades fastest of all: on co-edited files in the largest quartile it scores 0.113 against a random floor of 0.089, which is to say that in the repositories where retrieval matters most, knowing only which files are central is worth almost nothing. rff_hops_path never falls below sixth in any of the eight cells, and is fourth or better in five of them.

5 DISCUSSION

What we can and cannot say about the two keys. The tidy version of this paper would be that the import graph answers the compiler's question and the issue text answers the developer's, and that the field has been grading the first against the second. Our own controls will not support the tidy version, and it is worth being exact about what is left.

On the symbol key, structural methods win on the full corpus and are indistinguishable from issue-to-path matching once the population is matched to the co-edited key (Section 4.3). That key is also built by following imports, so we would have discounted a win there in any case. On the co-edited key, issue-to-path matching wins on the full corpus and loses to pure structure once

Table 15. Mean AUC (rank over all twenty-three orderings, oracle included) by quartile of repository size in graph nodes. $Q1 \leq 73$, $Q2 \leq 131$, $Q3 \leq 246$, $Q4$ up to 2,487.

Method	co_edited				symbol			
	Q1	Q2	Q3	Q4	Q1	Q2	Q3	Q4
rrf_pprpl_issue_path	.574 (2)	.487 (2)	.418 (2)	.369 (2)	.568 (3)	.476 (2)	.403 (5)	.375 (3)
rrf_hops_path	.552 (4)	.465 (4)	.397 (4)	.330 (5)	.562 (6)	.473 (3)	.409 (4)	.362 (5)
ppr_out_pl	.473 (16)	.359 (17)	.268 (16)	.168 (17)	.562 (5)	.450 (7)	.414 (3)	.372 (4)
ppr_und_pl	.497 (13)	.399 (9)	.316 (11)	.241 (12)	.568 (4)	.467 (4)	.423 (2)	.387 (2)
hops_lines	.517 (7)	.412 (7)	.329 (9)	.244 (11)	.569 (2)	.454 (6)	.402 (6)	.343 (8)
path_issue	.537 (6)	.437 (5)	.372 (6)	.302 (6)	.515 (14)	.416 (10)	.324 (13)	.257 (15)
bm25_issue	.516 (8)	.402 (8)	.334 (7)	.287 (7)	.476 (18)	.367 (18)	.273 (18)	.233 (17)
pagerank	.442 (21)	.299 (22)	.199 (21)	.113 (22)	.521 (11)	.370 (17)	.333 (12)	.286 (13)

tasks whose issues name a key file in code are removed (Section 4.10). Each half of the reversal, in other words, has an explanation that is not about the signals at all — one is a population effect, the other a leakage effect. The uncontrolled comparison reports both halves as findings about the signals; the controlled comparison does not.

Calling the first of these an artefact would undersell it. Single-file pull requests are 454 of the 1,156 our symbol key covers, and they are real work, not a sampling accident. What the matching control shows is that structure’s advantage on the symbol key is *concentrated* in them: on those 454 the best directed walk scores 0.605 and the six best orderings are all structural, against 0.417 for the same walk on the matched population. That is a finding in its own right, and an intuitive one — when a change touches one file, what it needs is most likely to be what that file imports. It is not evidence that structure beats words in general, which is what the unmatched table appears to say.

What remains is weaker but real. The two keys still induce different rankings, and a method tuned on one is not thereby good on the other. But the honest summary of the single-signal comparison is that neither family dominates once both confounds are controlled, and the interesting result is elsewhere.

Two effects, not one. The result we would most like carried forward is Section 4.11, and its interest is that it separates two things the literature reports as one number: filename leakage, which is causal and large, and the fact that reports naming their files are better reports in other ways as well. Both halves matter here. Ignore the first and a lexical baseline looks stronger than it is; ignore the second and the stratified comparison looks like proof that issue text does nothing, which our own within-task experiment contradicts. We resist the second reading in particular: the null in the no-mention stratum bounds the issue’s contribution at about 0.03 AUC on 531 pull requests. It does not show it is zero.

Fusion is the safe choice, and now a confirmed one. Reciprocal-rank fusion of a length-aware walk with issue-to-path matching is the best non-oracle ordering under both keys on the matched population, where it beats the best pure lexical ordering by 0.055 and 0.067 and the best pure structural one by 0.097 and 0.039 at the repository level. Across 200 random repository splits a mechanical criterion selects it on the choosing half in 198, and in 195 of those it is the best non-oracle ordering on the half that did not choose it (Section 4.8), so the recommendation no longer rests only on the data that produced it.

The honest form of the claim is robustness rather than dominance. Under the joint control of Table 12 the fusion leads on co-edited files in every stratum but sits behind two length-aware walks

on symbol files, by margins that do not survive correction ($p_{\text{Holm}} = 1$). Across every population and stratum we examined, on either key, the fusion is never significantly worse than the best pure ordering and is often significantly better. That is what we would recommend on, and it is a weaker claim than “fusion wins”.

We were also able to test the obvious alternative explanation. The fusions are the only methods given both the seed and the issue, so “a fusion just gets more input” is the null for the whole result. The issue-free twins of Section 4.11 answer it: blending a second list does help a little by itself — `rrf_pprpl_seedpath` beats its own walk `ppr_und_pl` by 0.049 on co-edited files — but the issue-aware fusion beats the twin by a further 0.069 on the full corpus. Complementarity is real; most of it is the leakage.

The field's default key, restated carefully. Every graph-based localization system we could check (Table 1) is graded against the files the gold patch touched — our `co_edited` source. On that source the best purely structural ordering ranks tenth of the twenty-three orderings we score, behind matching issue words against file paths, a method with no graph, no model and no seed. That gap narrows and then reverses once filename leakage is removed, which is itself the point: the comparison those systems are implicitly making depends on a property of the issue corpus that none of them report.

Two further qualifications matter. Our task hands every structural method a correct seed file for free, whereas `LocAgent`, `CoSIL` and the rest must localize from the issue text alone; their task is harder, and a component that underperforms given a free seed might still earn its place without one. And they are LLM agents that also read the issue, so the graph is one component among several. We do not claim their advertised component fails to do its work. The narrower claim we will defend is that on the ground truth those systems themselves adopt, graph proximity is not obviously the strongest available signal even when handed the answer's neighbourhood, and that nobody has reported the comparison or the leakage rate.

Filename leakage deserves to be a standard control. Two tasks in five in this corpus have an issue that names a key file as code. That is not a benchmark defect — real reports really do contain stack traces — but it means any lexical baseline evaluated without the stratification is reporting a mixture of two very different regimes, and any structural method compared against such a baseline is being judged against a moving target. The control costs a regular expression over text the benchmark already ships. We think retrieval papers should report the rate and the stratified result as a matter of course, in the same way they report the corpus size.

What “within two hops” is worth. The two-hop neighbourhood earns its reputation on recall and loses it on precision: it contains seven of every eight needed files and also half the repository. Reporting the recall without the size is, on this corpus, reporting half a statistic.

What this means for an agent's context, and what it does not. The budget framing is not decoration: at a median 286,385 tokens, most of these repositories do not fit in the windows current systems run with, so selection is forced rather than optional. Two numbers bound what selection can achieve. The answer is a median 2.98% of the repository, so a well-chosen context could be some 34× smaller than the whole; and an oracle ordering fits the complete answer inside 8,000 lines for 97.5% of pull requests where our best method manages 58.5%.

We are deliberately not claiming that better selection raises repair rates. We did not run an agent, and a retrieval improvement need not survive contact with one. What the ceiling licenses is the weaker statement: a file absent from the context cannot be used, so the share of tasks whose answer fits the budget bounds any system operating under it. It does not license the claim that the remaining gap is reachable from the signals we have, only that it is not closed off by the tasks

themselves. Section 6 specifies the experiment that would convert the bound into a performance claim, in advance of running it.

A note on order versus selection. An ordering plus a budget is a selection, and it is the selection that matters here. We make no claim about the position of a file *within* a prompt, which is a different question with its own literature; every result in this paper concerns which files are read at all.

The human reading of the same result. The budget for a developer is attention rather than tokens, and the results have a plain reading there. Asked to fix the login page, a developer thinks of the authentication files partly because the words match and partly because the imports connect; our fusion result says those two routes are largely complementary, and our leakage result says that how much the words help depends on how explicitly the report was written. We report the token framing because it makes the budget concrete and measurable; we do not think the finding is confined to machines.

Recommendations. (1) Report retrieval results separately for at least one structural and one behavioural notion of relevance, and report the worst case across them. (2) Report, and stratify by, whether the query text names the answer files in code. (3) Match populations before comparing across keys; a key that only exists for multi-file changes does not cover the same tasks as one that does not. (4) Treat a query-independent repository prior, not random order, as the floor for structural methods — on symbol-style keys it reaches 83% of a personalised walk’s absolute AUC, or 67% of the walk’s margin over a random ordering, which is the fairer reading. (5) Where a graph-based tool orders context by structure alone, add a lexical signal from the issue text and make the structural side length-aware before fusing; where one already uses both, report them separately as well as combined. (6) Report cost in tokens and report the ceiling alongside the score, so a reader can tell a hard task from a poor ordering. (7) Cluster paired tests by repository; pull requests within a repository are not independent, and ignoring that inflates significance by many orders of magnitude. (8) Before attributing a gain to issue text, run the two controls of Section 4.11: delete the answer files’ names from the issue and re-score the same tasks, and compare against the same method with the issue replaced by a seed-derived query. On our corpus the first costs the best fusion 0.061 AUC and a path-matching baseline 0.110; the second shows that even after redaction, an issue that named files is worth +0.095 against +0.016 for one that named none.

6 A PROTOCOL FOR THE DOWNSTREAM TEST, FIXED IN ADVANCE

This paper measures retrieval, not repair. The obvious next question — whether a context assembled by one of these orderings lets a model actually fix the issue — requires running models, which we have not done. Rather than gesture at it as future work, we specify it here in enough detail to be executed or contradicted, in the same spirit as the design freeze that preceded this study (§3). We commit to reporting the result whichever way it falls.

Corpus. The 1,162 scored pull requests of this study. Every one carries a test specification from the benchmark suite it came from, naming the tests that must change from failing to passing and those that must keep passing, so resolution can be decided by execution rather than by patch similarity. We note that only 15 of them overlap SWE-bench Verified, so this is not a re-run of an existing leaderboard.

Conditions. Seven contexts per task, each truncated to the same token budget. A repair experiment has no evidence key — the model gets one context and either the tests pass or they do not — so every condition names one concrete ordering: (i) the seed file alone, a floor; (ii) seed plus ppr_und_pl, the best structural ordering by worst-case rank across keys; (iii) seed

plus path_issue, the best lexical ordering; (iv) seed plus rrf_pprpl_issue_path, the best fusion on the matched population and the one confirmed on held-out repositories; (v) seed plus rrf_hops_path, the best fusion by worst-case rank, carried as well because the two criteria select different fusions and we will not choose between them after the fact; (vi) seed plus rrf_pprpl_seedpath, the issue-free twin of (iv), so that the downstream test inherits the control of Section 4.11; (vii) the repository truncated to the budget in a fixed canonical order, which is what a system with no retrieval step does. Budgets of 16,000 and 64,000 tokens, chosen from Table 10 to sit either side of the point where most answers begin to fit.

Models. Two, of clearly different capability, so that the result is not a property of one model. A weak model that fails everywhere and a strong model that succeeds everywhere would both be uninformative; the pilot below exists to detect that before the full run.

Generation. A single API call per task and condition, with no tools and no retrieval available to the model. This is essential rather than incidental: a model that can search the repository defeats the budget, and the quantity under study is what a fixed context supports.

Outcome and analysis. A task is resolved if the generated patch applies and the specified tests pass. The comparison is paired across conditions on the same tasks, tested with McNemar’s test, Holm-corrected across the condition pairs. Because pull requests cluster within repositories, the primary analysis is a mixed-effects logistic regression with a random intercept per repository and condition as a fixed effect, with the McNemar result reported alongside; this follows the unit argument of Section 3.6 rather than the pull-request-level convention we abandoned there. We do not require the models to be reliable; we require the *difference between conditions* to be measurable, and the pairing makes the stochastic component common to all conditions.

Pilot and stopping rule. A 150-task pilot on the weaker model precedes the full run. If the oracle condition resolves under 10% or over 90% of pilot tasks, the budget is outside the informative range and is adjusted before the main run — and that adjustment is reported.

What would falsify the implication. If conditions (ii) to (vi) do not differ from (vii) at equal budget, then retrieval quality does not transfer to repair at these budgets, and the results of this paper should be read as being about retrieval alone. If (iv) does not beat (vi), the issue text does not earn its place downstream either, which would extend Section 4.11 from retrieval to repair. Both outcomes are publishable and we would report them.

7 THREATS TO VALIDITY

Construct. Neither evidence source is ground truth about what a developer needed to read; they are recoverable proxies with opposite biases, which is why we report both and never merge them. The co-edited key in particular conflates files that were *changed* with files that had to be *read*, and the symbol key is built by a resolver that follows import edges, so it is partly circular with the structural methods it favours. Concurrent work is assembling keys from human annotation and from agent trajectories; scoring even a subset of our corpus against such a key would test our two proxies against a third of a different kind, and is the single addition we would most want to make. Coverage per line assumes whole files are read, which overstates the cost for agents that read selectively. The execution-based source is absent (§3.3) and would be the natural third leg.

Internal. The graph builder resolves imports statically and misses dynamic imports and plugin registration. The symbol key is produced by a static analyser and inherits its resolution failures. Both biases favour structural methods. Issue-path leakage is measured and does substantially drive the co-edited result: it is reported in Section 4.10 as a finding rather than a reassurance, and the

corresponding claim is stated conditionally throughout. Our leakage classifier is a regular expression over issue text and will mislabel some mentions in either direction; we report three nested strata rather than one so that the conclusion does not rest on a single cut.

External. Python only, file granularity only, and pull requests drawn from benchmark suites that select for reproducible test-verified fixes — a population plausibly better documented than average, which may flatter the issue-text methods. The median repository here has 131 graph files and very large repositories are under-represented. The size split in Section 4.12 runs on the uncontrolled corpus, so it shows only that the pattern of Table 5 is stable across size quartiles rather than widening with size. We cannot argue that the bias runs in a harmless direction; we can only say that it does not appear to be a size effect.

Conclusion. Pull-request-level aggregation, repository-level paired tests, Holm correction within comparison families and bootstrapping over repositories guard against the main inflation risks. Effect sizes accompany every test because several differences are significant and small. The fusion family, the worst-rank criterion and the directed walks were all added after the design freeze, so they are exploratory in origin; Section 4.8 confirms the selection on repositories that played no part in it, which addresses the selection but not the fact that the family was chosen after seeing results.

8 CONCLUSION

We compared import-graph proximity and issue-text retrieval as reading orders for Python change tasks on 2,412 tasks from 1,162 pull requests across 112 repositories, under two evidence-based definitions of which files a change required. Graded as they come, the two definitions appear to give opposite answers: structural orderings win on the files a change references, lexical orderings on the files a developer changed alongside it. Most of this paper is about why that headline does not survive contact with controls. Matching the populations, since a co-edited key only exists for a multi-file change, removes the structural advantage on the symbol key — and locates it, informatively, in the 454 single-file pull requests the unmatched comparison quietly added. Removing the two tasks in five whose issue names a key file in code removes the lexical advantage on the co-edited key. Each control dissolves one half, and neither family dominates once both are applied together.

Pressing on that second control gives the paper’s main result, in two experiments that bracket the question. Deleting a key file’s name from an issue that names it, and changing nothing else, costs issue-to-path matching 0.110 AUC on the affected tasks and costs nothing where there was nothing to delete. Replacing the issue entirely with a seed-derived query costs 0.156 on those same tasks but only 0.016 on the rest. Filename leakage is therefore causal and accounts for about two fifths of the apparent advantage of issue text here; the rest is that reports which name their files are better reports. A number that does not separate the two is reporting their sum.

The practical recommendation still stands, in a narrower form. Reciprocal-rank fusion of a length-aware walk with issue-text-to-path matching is the best non-oracle ordering under both keys on the matched population, is selected on one half and best on the other in 195 of 200 random repository splits, and is never significantly worse than the best pure ordering under any control we apply. It costs a BM25 index over file paths and contents. The graph’s two-hop neighbourhood, the standard justification for structural retrieval, turns out to be half the repository, and an oracle fits the complete answer in 8,000 lines for 97.5% of pull requests where our best method reaches 58.5%.

We recommend that context-retrieval work for code change report per evidence source, report and stratify by whether the query names the answer, compare any issue-aware method against the

same method with a seed-derived query, match populations before comparing across keys, cluster its tests by repository, and use a repository prior rather than random order as its floor.

DATA AVAILABILITY

The replication package contains the import graphs at every base commit, the task definitions and evidence keys, the issue-token bags, the per-task orderings and coverage curves, the per-file token counts, and the analysis scripts that produce every number and figure in this paper. It is available to reviewers at submission, and will be archived at a permanent identifier on acceptance.

USE OF AI TOOLS

Claude (Anthropic) was used to write the analysis and method code, to run the experiments and to draft this manuscript. All numeric results are produced by scripts in the replication package and were copied mechanically from their output files; the author verified the design, the claims and the text. Bibliographic entries were drafted with AI assistance and then checked against arXiv, the ACL Anthology and the publisher listings on 2026-09-23; the evaluation section of every system in Table 1 was read directly rather than summarised from an abstract. That check corrected eight entries drafted from memory, including a journal name, an author order, three titles changed between preprint versions, and one volume recorded as authored that is in fact edited. The corrections are listed in the replication package.

REFERENCES

- [1] Aider. 2023. Building a Better Repository Map with Tree Sitter. <https://aider.chat/2023/10/22/repomap.html>. Blog post, 22 October 2023. Accessed 2026-09-23.
- [2] Reem Aleithan, Haoran Xue, Mohammad Mahdi Mohajer, Elijah Nnorom, Gias Uddin, and Song Wang. 2024. SWE-Bench+: Enhanced Coding Benchmark for LLMs. arXiv:2410.06992 [cs.SE]
- [3] Ibragim Badertdinov, Alexander Golubev, Maksim Nekrashevich, Anton Shevtsov, Simon Karasik, Andrei Andriushchenko, Maria Trofimova, Daria Litvintseva, and Boris Yangel. 2025. SWE-rebench: An Automated Pipeline for Task Collection and Decontaminated Evaluation of Software Engineering Agents. arXiv:2505.20411 [cs.SE]
- [4] Shawn A. Bohner and Robert S. Arnold (Eds.). 1996. *Software Change Impact Analysis*. IEEE Computer Society Press, Los Alamitos, CA.
- [5] Dong Chen, Shaoxin Lin, Muhan Zeng, Daoguang Zan, Jian-Gang Wang, Anton Cheshkov, Jun Sun, Hao Yu, Guoliang Dong, Artem Aliev, Jie Wang, Xiao Cheng, Guangtai Liang, Yuchi Ma, Pan Bian, Tao Xie, and Qianxiang Wang. 2024. CodeR: Issue Resolving with Multi-Agent and Task Graphs. arXiv:2406.01304 [cs.SE]
- [6] Zhaoling Chen, Robert Tang, Gangda Deng, Fang Wu, Jialong Wu, Zhiwei Jiang, Viktor Prasanna, Arman Cohan, and Xingyao Wang. 2025. LocAgent: Graph-Guided LLM Agents for Code Localization. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Association for Computational Linguistics, Vienna, Austria, 8697–8727. <https://doi.org/10.18653/v1/2025.acl-long.426> Introduces the Loc-Bench benchmark. arXiv:2503.09089.
- [7] Bogdan Dit, Meghan Revelle, Malcom Gethers, and Denys Poshyvanyk. 2013. Feature Location in Source Code: A Taxonomy and Survey. *Journal of Software: Evolution and Process* 25, 1 (2013), 53–95. <https://doi.org/10.1002/smr.567>
- [8] Kang He and Kaushik Roy. 2026. SWE-Adept: An LLM-Based Agentic Framework for Deep Codebase Analysis and Structured Issue Resolution. arXiv:2603.01327 [cs.SE]
- [9] Zhonghao Jiang, Xiaoxue Ren, Meng Yan, Wei Jiang, Yong Li, and Zhongxin Liu. 2025. Issue Localization via LLM-Driven Iterative Code Graph Searching. In *Proceedings of the 40th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. Preprint titled “CoSIL: Software Issue Localization via LLM-Driven Code Repository Graph Searching”; arXiv:2503.22424.
- [10] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. 2024. SWE-bench: Can Language Models Resolve Real-world GitHub Issues?. In *International Conference on Learning Representations (ICLR)*. arXiv:2310.06770. SWE-bench Lite is released with this benchmark.
- [11] Amy J. Ko, Brad A. Myers, Michael J. Coblenz, and Htet Htet Aung. 2006. An Exploratory Study of How Developers Seek, Relate, and Collect Relevant Information during Software Maintenance Tasks. *IEEE Transactions on Software Engineering* 32, 12 (Dec. 2006), 971–987. <https://doi.org/10.1109/TSE.2006.116>

- [12] Han Li, Letian Zhu, Bohan Zhang, Rili Feng, Jiaming Wang, Yue Pan, Earl T. Barr, Federica Sarro, Zhaoyang Chu, and He Ye. 2026. ContextBench: A Benchmark for Context Retrieval in Coding Agents. arXiv:2602.05892 [cs.SE]
- [13] Wei Liu, Chao Peng, Pengfei Gao, Aofan Liu, Wei Zhang, Haiyan Zhao, and Zhi Jin. 2025. GraphLocator: Graph-guided Causal Reasoning for Issue Localization. arXiv:2512.22469 [cs.SE]
- [14] Yingwei Ma, Qingping Yang, Rongyu Cao, Binhua Li, Fei Huang, and Yongbin Li. 2025. Alibaba LingmaAgent: Improving Automated Issue Resolution via Comprehensive Repository Exploration. In *Companion Proceedings of the 33rd ACM International Conference on the Foundations of Software Engineering (FSE Companion '25), Industry Track*. <https://doi.org/10.1145/3696630.3728549> arXiv:2406.01422 (v1 titled “How to Understand Whole Software Repository?”).
- [15] Siru Ouyang, Wenhao Yu, Kaixin Ma, Zilin Xiao, Zhihan Zhang, Mengzhao Jia, Jiawei Han, Hongming Zhang, and Dong Yu. 2025. RepoGraph: Enhancing AI Software Engineering with Repository-Level Code Graph. In *International Conference on Learning Representations (ICLR)*. arXiv:2410.14684.
- [16] Bowen Qin and Yi Xie. 2026. Agent Retrieval Bench: Evaluating Repository Context Retrieval for Coding Agents. arXiv:2607.24882 [cs.SE]
- [17] Revanth Gangi Reddy, Tarun Suresh, JaeHyeok Doo, Ye Liu, Xuan Phi Nguyen, Yingbo Zhou, Semih Yavuz, Caiming Xiong, Heng Ji, and Shafiq Joty. 2025. SweRank: Software Issue Localization with Code Ranking. arXiv:2505.07849 [cs.SE]
- [18] Stephen Robertson and Hugo Zaragoza. 2009. The Probabilistic Relevance Framework: BM25 and Beyond. *Foundations and Trends in Information Retrieval* 3, 4 (2009), 333–389. <https://doi.org/10.1561/15000000019>
- [19] Martin P. Robillard. 2008. Topology Analysis of Software Dependencies. *ACM Transactions on Software Engineering and Methodology* 17, 4 (Aug. 2008), 18:1–18:36. <https://doi.org/10.1145/13487689.13487691>
- [20] Lintang Sutawika, Aditya Bharat Soni, Bharath Sriraam R R, Apurva Gandhi, Taha Yassine, Sanidhya Vijayvargiya, Yuchen Li, Xuhui Zhou, Yilin Zhang, Leander Melroy Maben, and Graham Neubig. 2026. CodeScout: An Effective Recipe for Reinforcement Learning of Code Search Agents. arXiv:2603.17829 [cs.SE]
- [21] Chunqiu Steven Xia, Yinlin Deng, Soren Dunn, and Lingming Zhang. 2025. Demystifying LLM-Based Software Engineering Agents. *Proceedings of the ACM on Software Engineering* 2, FSE (July 2025), FSE037. <https://doi.org/10.1145/3715754> Agentless. arXiv:2407.01489.
- [22] Chengxing Xie, Bowen Li, Chang Gao, He Du, Wai Lam, Difan Zou, and Kai Chen. 2025. SWE-Fixer: Training Open-Source LLMs for Effective and Efficient GitHub Issue Resolution. In *Findings of the Association for Computational Linguistics: ACL 2025*. 1123–1139. <https://doi.org/10.18653/v1/2025.findings-acl.62> arXiv:2501.05040.
- [23] Boyang Yang, Jiadong Ren, Shunfu Jin, Yang Liu, Feng Liu, Bach Le, and Haoye Tian. 2025. Enhancing Repository-Level Software Repair via Repository-Aware Knowledge Graphs. arXiv:2503.21710 [cs.SE] KGCompass. An earlier version was titled “KGCompass: Knowledge Graph Enhanced Repository-Level Software Repair”.
- [24] Annie T. T. Ying, Gail C. Murphy, Raymond Ng, and Mark C. Chu-Carroll. 2004. Predicting Source Code Changes by Mining Change History. *IEEE Transactions on Software Engineering* 30, 9 (Sept. 2004), 574–586. <https://doi.org/10.1109/TSE.2004.52>
- [25] Linghao Zhang, Shilin He, Chaoyun Zhang, Yu Kang, Bowen Li, Chengxing Xie, Junhao Wang, Maoquan Wang, Yufan Huang, Shengyu Fu, Elsie Nallipogu, Qingwei Lin, Yingnong Dang, Saravan Rajmohan, and Dongmei Zhang. 2025. SWE-bench Goes Live! arXiv:2505.23419 [cs.SE] NeurIPS 2025 Datasets and Benchmarks Track (per the authors’ repository).
- [26] Shaoqiu Zhang, Yuhang Wang, Jialiang Liang, Yuling Shi, Wenhao Zeng, Maoquan Wang, Shilin He, Ningyuan Xu, Siyu Ye, Kai Cai, and Xiaodong Gu. 2026. SWE-Explore: Benchmarking How Coding Agents Explore Repositories. arXiv:2606.07297 [cs.SE]
- [27] Thomas Zimmermann, Peter Weißgerber, Stephan Diehl, and Andreas Zeller. 2004. Mining Version Histories to Guide Software Changes. In *Proceedings of the 26th International Conference on Software Engineering (ICSE)*. IEEE Computer Society, Edinburgh, United Kingdom, 563–572.